Advancing CMA-ES with Learning-Based Cooperative Coevolution for Scalable Optimization

Anonymous Author(s)

Abstract

Recent research in Cooperative Coevolution (CC) has achieved promising progress in solving large-scale global optimization problems. However, existing CC paradigms have a primary limitation in that they require deep expertise for selecting or designing effective variable decomposition strategies. Inspired by advancements in Meta-Black-Box Optimization, this paper introduces LCC, a pioneering learning-based cooperative coevolution framework that dynamically schedules decomposition strategies during optimization processes. The decomposition strategy selector is parameterized through a neural network, which processes a meticulously crafted set of optimization status features to determine the optimal strategy for each optimization step. The network is trained via the Proximal Policy Optimization method in a reinforcement learning manner across a collection of representative problems, aiming to maximize the expected optimization performance. Extensive experimental results demonstrate that LCC not only offers certain advantages over state-of-the-art baselines in terms of optimization effectiveness and resource consumption, but it also exhibits promising transferability towards unseen problems.

CCS Concepts

• **Computing methodologies** → **Sequential decision making**; *Continuous space search.*

Keywords

CMA-ES, Large-scale Global Optimization, Reinforcement Learning

ACM Reference Format:

Anonymous Author(s). 2026. Advancing CMA-ES with Learning-Based Cooperative Coevolution for Scalable Optimization. In . ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Black box optimization (BBO) is a class of optimization problems whose objective function is either unknown or too intricate to be mathematically formulated [37]. Consequently, BBO requires interaction-based information acquisition without access to underlying mathematical expressions or gradients. Within the context of BBO, Large-Scale Global Optimization (LSGO), which involves

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

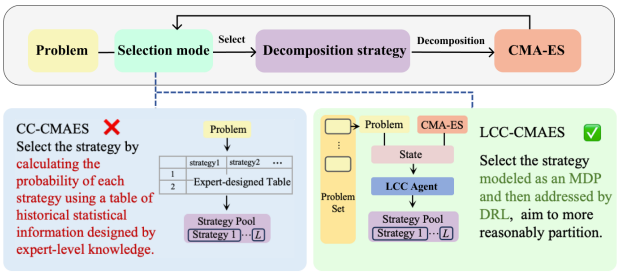

Figure 1: The core idea of LCC-CMAES.

thousands to tens of thousands of variables, has numerous real-world applications [2, 9, 10] to drive resource savings, cost control, and efficiency enhancement [14, 32, 48, 84]. Many works have proposed LSGO variants of algorithms originally applied to lower-dimensional BBO problems, such as Sep-CMAES [54], LM-MA-ES [35], and so on [1, 20, 29, 34], to tackle such problems. Besides, Persistent Evolution Strategies (PES) [73], presented in an outstanding paper at ICML-2021, combines ideas from gradient-based optimization with evolutionary strategies to improve optimization efficiency and accuracy. However, the "curse of dimensionality" represents a significant challenge for such problems: as the number of variables increases, the complexity of optimization grows exponentially, necessitating extensive iterations for exploration [18].

To address LSGO, inspired by the divide-and-conquer philosophy, a framework named Cooperative Co-evolution (CC) first divides the variables into several subgroups, then optimize these subgroups (considered as lower-dimensional BBO problems) using Evolutionary Algorithms (EAs), and finally integrates them into a comprehensive global optimization solution [7, 21, 52, 81]. In the CC framework, an important issue is placing non-separable variables within the same subgroup to accurately divide the problem dimensions, which is so called decomposition strategy [53, 72]. The researchers initially tried random decomposition and some decomposition strategies utilizing statistical data but did not obtain satisfactory results [52, 72]. Later, they attempted to dynamically select strategies by calculating the probability of each using a table of historical statistical information, designed by expert-level knowledge, which yielded some positive effects (e.g., CC-CMAES [33]). Furthermore, the researchers designed a series of decomposition strategies based on expert-level knowledge to more accurately identify variable interactions for precise decomposition, but this precise decomposition led to substantial additional function evaluations (FEs) costs [50, 65, 68]. According to the above, a primary limitation in the current CC framework is the *Expert-Level Knowledge Dependency*: these decomposition strategies are based on hand-crafted rules, heavily reliant on expert-level optimization knowledge and might not be generalizable towards unseen problems. Therefore, considering methods that do not require expert-level knowledge

for decomposition could be a more suitable solution for tackling challenging real-world problems.

- LCC is designed to create an intelligent decision-making agent that autonomously selects effective decomposition strategies tailored to various problem environments and optimization states. We have formulated this process as a Markov Decision Process (MDP) and utilized DRL to construct the agent. This approach replaces traditional, expert-designed selection modes, marking a significant advance in automating and optimizing the decomposition strategy within the CC frameworks for large-scale BBO.
- Taking the Covariance Matrix Adaptation Evolution Strategy (CMA-ES) as the underlying optimizer, we develop the LCC-CMAES algorithm. Figure 1 shows the core idea of LCC-CMAES. We have designed a set of straightforward yet representative statistical features to capture essential grouping information and reflect the optimization state. Based on the state, LCC selects an appropriate decomposition strategy from a strategy pool of random decomposition (RD), Min-Variance decomposition (MiVD) and Max-Variance decomposition (MaVD) - enhancing the efficacy of CMA-ES.
- We conducted detailed comparisons with various leading LSGO algorithms in more challenging settings to illustrate the limitations in practical problems. The experimental results demonstrate that LCC-CMAES excels not only in terms of resource consumption but also in optimization results compared to other algorithms. Additionally, LCC-CMAES exhibits transferability, showing outstanding performance on other unseen problem sets after training.

The remainder of this paper is organized as follows: Section 2 discusses related work. Section 3 provides the preliminary knowledge necessary for understanding CC-CMAES and MDP. Section 4 describes the overall architecture of LCC, as well as the specific design of its MDP and network. Section 5 presents the experimental results and provides a detailed analysis. Finally, Section 6 concludes the paper and outlines future work.

To alleviate the burdensome task of manual fine-tuning with expert-level knowledge, recent research has proposed the concept of Meta-Black-Box Optimization (MetaBBO) [25, 27, 28, 37, 38, 45]. This paradigm has showcased the power of leveraging deep reinforcement learning (DRL) in a data-driven fashion at the meta-level to mitigate expert-level knowledge of low-level black-box optimizers. Numerous studies have shown that MetaBBO enables the black-box optimizers to achieve more effective optimization performance through enhanced parameter configuration [36, 64, 76], algorithm/operator selection [16, 31], and update rule generation [4, 8, 23, 82]. Inspired by MetaBBO, we introduce **L**earning-Based **C**ooperative **C**oevolution (LCC), a pioneering framework that dynamically schedules decomposition strategies without expertise during optimization processes. The main contributions of this work are summarized as follows:

2 Related Works

As mentioned earlier, our LCC is inspired by MetaBBO and operates within the CC framework. Therefore, in this section, we will review MetaBBO and several important problem decomposition strategies under the CC framework.

2.1 MetaBBO

To alleviate the burdensome task of manual fine-tuning, the concept of MetaBBO has been proposed by recent research [6, 11, 37, 38, 59, 80]. MetaBBO aims to refine black-box optimizers by identifying optimal configurations or parameters through an automatic decision process without requiring expertise, thereby boosting overall performance across various problem instances within a given problem domain. MetaBBO-RL is one of approaches of MetaBBO [17, 39, 40, 60, 66], which models the optimizer fine-tuning as a MDP and learns an RL agent to automatically make decisions without expertise. The meta-objective of MetaBBO-RL is to learn a policy (RL agent) Π^* that maximizes the expectation of the accumulated meta-performance improvement r_t (also called reward) over the problem set distribution ξ, $\mathbb{E}_{\upsilon\sim\xi,\Pi^*}\left[\sum_{t=0}^{T} r_t\right]$, where T denotes the all times of making decisions and υ is the problem of problem set Υ. Specifically, in the aspect of operator selection, MetaBBO-RL automates the tuning process, significantly reducing the time and expertise needed to customize algorithms for specific unseen problems, while also potentially enhancing overall optimization performance [15, 74, 75, 83]. This has been confirmed in numerous research studies: RL-DAS [16], based on MetaBBO-RL, selects operators for Differential Evolution algorithms, leveraging their complementary strengths to enhance optimization performance and demonstrating favorable generalization across different problem classes; RLDMDE [79] employs RL so that each subpopulation can adaptively select a mutation strategy based on the current environmental state (population diversity), thereby boosting the self-adaptation of subpopulations; similarly, RLEMMO [30], the first generalizable MetaBBO-RL framework for solving multimodal optimization problems (MMOP), selects operators for search strategies, directly addresses unseen problems, and achieves competitive optimization performance in both quality and diversity against several strong MMOP solvers.

2.2 CC and the Problem Decomposition Strategies

Inspired by the "divide and conquer" philosophy, CC is a framework to solve LSGO by the decomposition-based approach[48, 49]. It first divides the variables into several subgroups, then optimize these subgroups using EAs, and finally integrates them into a global optimization solution.

CCGA [52] is the first strategy to use CC for problem decomposition, splitting an D-dimensional problem into D one-dimensional problems, where D is the dimensionality of the problem. However, both practical tests [52] and theoretical analyses [72] have suggested that completely decomposing into one-dimensional problems poses a risk of introducing spurious minima. To mitigate this issue, strategies such as k-s dimensional decomposition and bipartite decomposition have been proposed [61, 72], but these algorithms do not take into account the structure of the problem or interactions between variables, potentially placing interacting variables in different components, which adversely affects optimization performance [48]. To achieve more precise decomposition,

researchers have started from the definition of separability, defining various types of separability such as additive separability [26], multiplicative separability [24], and composite separability [68], and have developed a range of variable interaction identification algorithms, such as DG2 [50], RDG [65], GDG [41] and CSG [68]. In addition, researchers have further studied how to accurately decompose overlapping variables, such as DOV [43], OCC [22], and OEDG [67]. However, the cost of improving accuracy in this way includes a large number of expert-designed separability methods and additional FEs. Strategies based on probabilistic and statistical methods do not have these issues. They perform multiple rounds of grouping optimization before forming the final optimization result to capture problem structure and variable interactions [81]. Relying on expertise, many algorithms were proposed [55, 62, 69, 70], such as the Delta method [47] based on theory that the improvement intervals for inseparable variables are relatively smaller than those for separable variables [56], the Fitness Difference Minimization (FDM) method exemplified by DIMA [57] and CC-CMAES [33] based on covariance matrices and expert-designed selection mode. Besides, contribution-based decomposition methods [77], such as CCFR [78], DCC [85], and CBCCO [21], represent another novel strategy. Although these two strategies do not have the additional FEs, they rely on expertise, so in different scenarios, they may fail to meet the requirements for reasonable decomposition[48, 53]. Therefore, considering methods that do not require expert-level knowledge for decomposition might be a more suitable solution for more challenging real-world problems.

3 Preliminaries

3.1 CC-CMAES

Covariance Matrix Adaptation Evolution Strategy (CMA-ES) [19] is a representative EA that operates by repeatedly sampling offspring according to a distribution and updating the distribution with the performance of the sampled offspring until a stopping criterion is met (e.g., reaching the total number of generations TG).

$$x_{(g+1)}^{(k)} \sim N\left(\omega_{(g)}, \sigma_{(g)}^2 \cdot C_{(g)}\right), \quad k = 1, 2, \cdots, \lambda \tag{1}$$

Equation (1) shows the sampling process in a population P with offspring size λ at generation g. $\omega_{(g)} \in \mathbb{R}^D$, $C_{(g)} \in \mathbb{R}^{D\times D}$, and $\sigma_{(g)} \in \mathbb{R}$ are the Gaussian mean, covariance matrix, and global step size, respectively, at generation g. CC-CMAES [33] uses the CC framework with CMA-ES, featuring three decomposition strategies: Min-Variance Decomposition (MiVD), Random Decomposition (RD), and Max-Variance Decomposition (MaVD), ranging from exploitative to exploratory. It dynamically selects one strategy to optimize subgroups with CMA-ES for a fixed number of generations until termination criteria are met. MiVD, RD, and MaVD decompose the space based on the rank of the diagonal of the covariance matrix. MiVD sequentially selects D/m variables following the rank order to minimize the diversity among their variances. In contrast, MaVD selects one variable, then skips D/m variables to select the next variable each time, which maximizes diversity. RD randomly selects D/m variables within each subspace. The subspace covariance matrix $C_{sub_i} \in \mathbb{R}^{(D/m)\times(D/m)}$ and mean $\omega_{sub_i} \in \mathbb{R}^{D/m}$ are extracted from the global covariance matrix C and mean ω as $C_{sub_i} = C[subdims_i, subdims_i], \omega_{sub_i} = \omega[subdims_i]$, where $subdims_i \in [1, D]^{D/m}$ represents the dimension index set of subgroup $i \in [1, \ldots, m]$. C and ω are updated using C_{sub_i} and ω_{sub_i} is its inverse process.

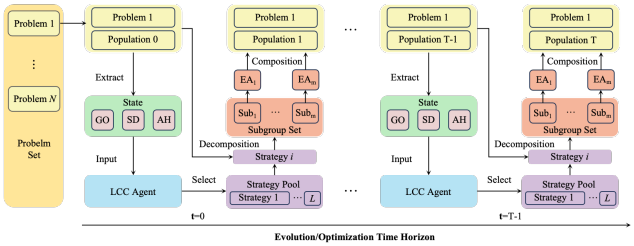

Figure 2: The overall structure of LCC.

3.2 Markov Decision Process

A Markov Decision Process (MDP) is commonly characterized as $\mathcal{M} :=< \mathcal{S}, \mathcal{A}, \mathcal{T}, R >$. At each time step t, given the current environment state $s_t \in \mathcal{S}$, an action $a_t \in \mathcal{A}$ is performed according to a policy $\Pi : \mathcal{S} \rightarrow \mathcal{A}$. Then the environment reaches at the next state s_{t+1} according to the transition dynamics $\mathcal{T}(s_{t+1} \mid s_t, a_t)$. The reward function $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ indicates the feedback r_t from the environment. Given a finite horizon (suppose T steps) of interactions with the environment, a sampled trajectory is defined as $\tau := (s_0, a_0, s_1, \cdots, s_T)$. Then an MDP is solved by finding an optimal policy π^* that maximizes the expected accumulated rewards over all possible trajectories:

$$\pi^* = \arg\max_{\pi\in\Pi} \mathbb{E}_{\tau\sim\pi(\tau)}\left[\sum_{t=0}^{T} \gamma^{t-1} r_t\right] \tag{2}$$

where $\pi(\tau)$ denotes the sampling probability of τ and γ is a predefined discount factor. In the context of DRL [44], the policy π is parameterized with a neural network π_θ, which makes the gradient based learning methods (e.g., PPO [58]) available for searching the optimal policy.

4 Methodology

4.1 LCC Overview

LCC primarily consists of three main components: a problem set Υ with N problems, a CC decomposition strategy pool Λ, and an underlying EA optimizer (e.g., CMA-ES). Λ includes a variety of strategies chosen from existing decomposition strategies. The detailed architecture, illustrated in Figure 2, can be conceptualized as an MDP. In an MDP, as introduced in Equation (2), multiple elements are fundamental, such as state $s_t \in \mathcal{S}$, action $a_t \in \mathcal{A}$, and reward $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$. The DRL agent targets at the optimal policy π^* that select an appropriate decomposition strategy in Λ to maximize the expected accumulated reward over all the problems $\upsilon \in \Upsilon$ as $\pi^* = \arg\max_{\pi\in\Pi} \frac{1}{N}\sum_{k=1}^{N}\sum_{t=0}^{T}\gamma^{t-1}R(s_t, a_t|\upsilon)$. First, a problem υ is selected from the problem set Υ. For this problem, we analyze Global Optimization (GO) information, Subgroup Decomposition (SD) information, and Action History (AH) information using Exploratory Landscape Analysis (ELA) [42] and Fitness Landscape Analysis (FLA) [51]. This analysis is used to design the state to ensure it contains sufficient information to select an appropriate

decomposition strategy. Based on the state s_t, LCC selects a decomposition strategy from the CC decomposition strategy pool Λ to decompose the problem and subsequently optimize each subgroup using the underlying EA optimizer. A corresponding reward is designed to reflect improvements in the MDP $\mathcal{M}$. Finally, the optimized subgroups are combined to form a new global population, completing an epoch. With CMA-ES as the underlying optimizer, we instantiate LCC, naming it the LCC-CMAES algorithm. Using LCC-CMAES as a concrete example, we will describe the specific design of the MDP and the network.

4.2 MDP Formulation

4.2.1 State. The state space of LCC-CMAES encompasses the Global Optimization (GO) information $s_{\text{GO}} \in \mathbb{R}^{12}$, Subgroup Decomposition (SD) information $s_{\text{SD}} \in \mathbb{R}^{4\times m}$, and Action History (AH) information $s_{\text{AH}} \in \mathbb{R}^{2\times L}$. Details are shown in the Table 1.

For s_{GO}, reflects the CMA-ES state and global optimization state, revealing the complexity and difficulty of the optimization problem as well as the relationships between various dimensions. For s_{SD}, we have designed four types of features based on probabilistic and statistical methods within the CC framework to reflect the variable grouping status within a subgroup, which provides detailed insights into the dynamics of variable relationships and optimization progress in the subgroups. For s_{AH}, given that LCC includes a CC decomposition strategy pool Λ, we derive s_{AH} to provide the RL agent with additional contextual knowledge about the optimization capabilities of the candidate strategies.

Finally, the complete state in the MDP of LCC-CMAES is the integration of s_{GO}, s_{SD} and s_{AH}.

$$\text{state} := \left\{ s_{\text{GO}} \in \mathbb{R}^{12}, s_{\text{SD}} \in \mathbb{R}^{4\times m}, s_{\text{AH}} \in \mathbb{R}^{2\times L} \right\} \tag{3}$$

Here, m represents the number of subgroups (where m is 10 in LCC-CMAES), and L denotes the number of CC decomposition strategies in the pool (where L is 3 in LCC-CMAES).

4.2.2 Action. We designed a strategy pool Λ in advance, containing various decomposition strategies for selection. LCC selects a CC decomposition strategy from Λ based on the state to achieve dynamic decomposition. For the purpose of balancing exploration and exploitation, LCC-CMAES utilize three types of decomposition strategies [33]: MiVD, RD, and MaVD as introduced in Section 3.1. This operation is represented as an integer, which indicates the index of the chosen strategy within the strategy pool of L candidate strategies, denoted as $a \in [1, L]$. Next, based on the selected strategy, the problem is divided into smaller-dimensional subproblems and then optimized using CMA-ES. The optimization results of each subproblem are subsequently combined into a global optimization result.

4.2.3 Reward. To guide the agent towards achieving a lower cost, the reward function should consider the absolute reduction in cost at each time step t:

$$r_t = \frac{f^*_{t-1} - f^*_t}{f^*_0 - f^*} \tag{4}$$

where f^*_{t-1} and f^*_t are the global best fitness in the t-1 step and the t step. f^* is the optimal fitness of the problem, f^*_0 is the global best fitness in the initial population, which serves as a normalization factor. This measures the performance improvement brought in the step t optimization.

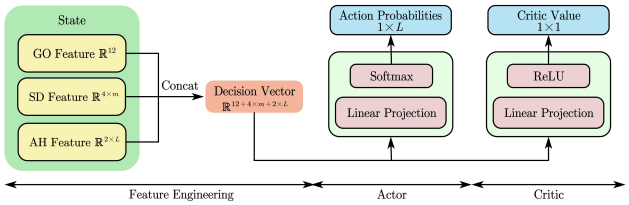

Figure 3: The Neural Network workflow for π_θ (Actor) and v_ϕ(Critic).

4.3 Network Design

As shown in Figure 3, the network consists of three modules: Feature Processing, Actor, and Critic. s_{GO}, s_{SD}, and s_{AH} are first fused to form a state representation vector DV. Based on this representation, the Actor outputs the probability distribution of candidate strategies, while the Critic estimates the return value.

The Actor decides the probability for selecting a strategy from the CC decomposition strategies pool. As mentioned in Section 4.2.1, LCC-CMAES has $m = 10$ subgroups and $L = 3$ actions. We first concatenate $s_{\text{GO}} \in \mathbb{R}^{12}$, $s_{\text{SD}} \in \mathbb{R}^{4\times m}$ and $s_{\text{AH}} \in \mathbb{R}^{2\times L}$ to generate the Decision Vector $DV \in \mathbb{R}^{58}$ as $DV = s_{\text{GO}} \oplus s_{\text{SD}} \oplus s_{\text{AH}}$. Then we map DV to a three-layer Multi-Layer Perceptron (MLP) network with the structure $(58 \times 64 \times 64 \times L)$, cooperating with a ReLU [46] activation after each hidden layer. Following the Softmax operation, the Actor outputs a probability distribution over the strategy pool Λ, which is then used to sample the strategy.

The Critic also takes DV as input and uses the same MLP structure as Actor, where the output dimension is set to be 1 for critic value prediction. However, their MLP parameters are not shared, and the training is conducted independently.

4.4 Workflow

LCC-CMAES's workflow begins with selecting a problem υ from the problem set Υ, initializing the global dimension D, global covariance matrix $C_0 = I$, global population P_0, global step size $\sigma_0 = radius$ and global mean vector ω_0. Then training for υ starts and terminates when $MaxFEs$ is exhausted or the global best fitness $gbest_t$ is lower than the termination error. After initialization, an MDP starts. At step t, state s_t can be calculated by following Table 1. Based on s_t, the Actor policy π_θ with parameters θ takes the Decision Vector s_t as input and outputs the probability distribution of candidate strategies $\pi(a_t|s_t)$, while the critic network v_ϕ with parameters ϕ predicts the expected return values (accumulated rewards) of s_t. Once the strategy is determined, the problem is decomposed into subgroups, marking the end of the CC problem decomposition layer and transitioning into the subgroup optimization layer. Each subgroup is optimized using CMA-ES until $SubMaxFEs$ is reached, with σ_t updated by the offspring in each subgroup. Once all subgroup optimization completed, σ_t is updated to σ_{t+1}, C_t, ω_t, and P_t are updated to C_{t+1}, ω_{t+1}, and P_{t+1}, using on the C_{sub_i} and ω_{sub_i} obtained from each subgroup, as mentioned in Section 3.1. At this point, state s_{t+1} can be calculated by following Table 1. Then the reward r_t is observed. The trajectories of states s_t, actions a_t, and rewards r_t are

Table 1: State features.

Feature	Index	Formula	Explain	Feature	Index	Formula	Explain
s_{GO}	1	$\text{Max}(\frac{\omega_t}{radius})$	Max/Min/Mean of global mean ω_t normalized by $radius$. ω_t is the global mean at step t, and $radius$ is half the bound range. Reflects population status and CMA-ES state.	s_{SD}	13-22	$\text{Mean}(\text{Corrcoef}(C_{sub_i}))$	Mean correlation within subgroup i via covariance matrix C_{sub_i}.
	2	$\text{Mean}(\frac{\omega_t}{radius})$			23-32	$\text{Mean}(\frac{\Delta subpop_i}{\lambda \times radius})$	$\Delta subpop_i$: difference between first and last generations. Based on Delta method (Sec 2.2) to reveal variable interactions.
	3	$\text{Min}(\frac{\omega_t}{radius})$			33-42	$\text{Mean}(\frac{\text{Var}(subpop_i)}{radius^2})$	Variance of subgroup generations. Reflects dimensional volatility and exploration/exploitation balance.
	4	$\text{Max}(\text{Corrcoef}(C_t))$	Correlation matrix from global covariance C_t. Reflects variable dependencies and CMA-ES status.		43-52	$\frac{d_{max_i}}{diameter}$	Max distance in subgroup population d_{max_i} normalized by diameter. Describes convergence.
	5	$\text{Mean}(\text{Corrcoef}(C_t))$		s_{AH}	53-55	$\frac{\sum(\Delta r_j)}{num_j}$	Mean reward difference ($\Delta r_j = r_t - r_{t-1}$) for action j. num_j is selection count. Measures algorithm j's contribution.
	6	$\text{Min}(\text{Corrcoef}(C_t))$			56-58	$\frac{\sum(\Delta gbest^{(j)})}{2 \times radius \times num_j}$	Euclidean norm of the change in $gbest^{(j)}$. Reflects algorithm j's effectiveness.
	7	$\frac{\sigma_t}{radius}$	Normalized global step size σ_t indicating exploration and exploitation conditions.				
	8	$\text{Max}(\frac{gbest_t}{radius})$	Normalized global best point $gbest_t$. Tracks the optimization state within the problem domain.				
	9	$\text{Mean}(\frac{gbest_t}{radius})$					
	10	$\text{Min}(\frac{gbest_t}{radius})$					
	11	$\frac{f_t^*}{f_{t-1}^*}$	Ratio of best fitness between steps. Reflects incremental optimization effects.				
	12	$\frac{FEs}{MaxFEs}$	Ratio of remaining to max evaluations. Tracks budget consumption.				

recorded and then used by the PPO method to train the policy net π_θ and the critic net v_ϕ for K times after the completion of optimization. PPO is trained in an actor-critic manner. It proposes a novel objective with clipped probability ratios, which forms a first-order estimate (i.e., lower bound) of the policy's performance. Its objective function at the k-th learning iteration ($k \in [1, K]$) is defined as: $L_\pi(\theta^{(k)}) := \mathbb{E}\left[\min\left(\eta(\theta^{(k)})\hat{A}, \text{clip}(\eta(\theta^{(k)}), 1-\epsilon, 1+\epsilon)\hat{A}\right)\right]$ where $\eta^{(k)} := \frac{\pi_{\theta^{(k)}}(a_t|s_t)}{\pi_{\theta^{(0)}}(a_t|s_t)}$ is the ratio of the probabilities under the current policy and the old policy before the K-step learning process, performing the importance sampling. $\hat{A}$ is the estimated advantage calculated as the difference between the target return G and the estimated return $\hat{G}$. Using $L_\pi(\theta)$, the gradients are back-propagated through the network to update the parameters and achieve the training effect. The critic network v_ϕ takes the Decision Vector as input and outputs a critic value prediction to estimate the return value $\hat{G}$. The loss function of the critic network v_ϕ is: $L_v(\phi) := \text{MSE}(G, \hat{G})$.

5 Experiments

5.1 Experimental Setup

5.1.1 Comparison Algorithms for LCC-CMAES . For a comprehensive comparison, we selected CC-CMAES [33], CSG [68] , and FII [13] as the comparison algorithms under CC framework. We then selected LSGO baselines without CC: CMA-ES [19], Sep-CMAES [54], LM-MA-ES [35]. Besides, MetaBBO method MetaES [23] and local search method L-BFGS [3] were also chosen. Among the comparison algorithms under CC framework, CC-CMAES, having the same decomposition strategy pool, is introduced in Section 3.1. CSG, FII are the algorithms that need addition FEs costs for decomposition: CSG is currently the most powerful multi-stage variable identification algorithm; FII is a rapid identification algorithm that reduces the FEs consumed by decomposition . These algorithms under CC framework all use CMA-ES as the underlying optimizer. Among other types of comparison algorithms, MetaES discovers evolutionary strategies via MetaBBO and serves as the global optimization comparison algorithm within MetaBBO; L-BFGS is an optimization algorithm from outside the evolutionary algorithms community, and it is widely used in practical applications, especially for LSGO.

5.1.2 Benchmark and Hyperparameter Settings. The CEC 2013 LSGO benchmark [26] comprises a total of $N = 15$ problems, which are divided into five types of functions: Fully-separable Functions (F1-F3), Partially Additively Separable Functions with a separable subcomponent (F4-F7), Partially Additively Separable Functions with no separable subcomponents (F8-F11), Overlapping Functions (F12-F14), and Non-separable Function (F15). Additionally, we partitioned the CEC 2013 LSGO benchmark suite, as shown in Table 2. An asterisk "*" marks the problems used for training, while the rest were used for testing. Except for the Non-separable Function, each category has training problems, and F1, F4, and F8 are variants of the Elliptic Function; F5 and F9 are variants of the Rastrigin Function. This allows for testing LCC-CMAES's generalizability on unseen functions within the same type, with the Non-separable Functions tested as the unseen functions and type.

The hyperparameter settings in this paper are as follows: the total number of generations (TG) is 50, the offspring size (λ) is 20, subgroup maximum function evaluations ($SubMaxFEs$) is 1E3, the number of subgroups (m) is 10, learning rate (lr) is 6E-4, number of epochs ($Epoch$) is 90, Mini PPO iterations (K) is 3, the problem's dimension (D) is 1E3 (F13 and F14 in CEC2013LSGO is 905), the number of action selecting (ns) is 20. It's worth noting that, to realistically address the practical problems and accommodate the extra decomposition costs required by various decomposition strategies, we set $MaxFEs = TG \times \lambda \times m \times ns = 50 \times 20 \times 10 \times 20 = 2\text{E}5$. The settings of the comparison algorithms remain the same as those in their original papers.

Table 2: Comparing LCC-CMAES with comparison algorithms on CEC 2013 LSGO.

problem	Algorithms under CC Framework				Algorithms without CC				
	LCC-CMAES	CC-CMAES	CSG	FII	CMA-ES	Sep-CMAES	LM-MA-ES	MetaES	L-BFGS
*1	2.045E+07 ±2.781E+06	3.244E+08(+) ±1.112E+08	4.182E+08(+) ±3.882E+07	4.227E+08(+) ±2.231E+07	4.223E+08(+) ±3.827E+07	**8.591E+06(-) ±1.777E+06**	4.691E+07(+) ±2.973E+06	2.420E+11(+) ±2.351E+10	9.085E+09(+) ±1.396E+08
2	**1.349E+03 ±3.412E+02**	2.159E+03(+) ±4.125E+02	2.636E+03(+) ±1.393E+02	2.592E+03(+) ±9.076E+01	5.108E+03(+) ±2.32E+02	5.414E+03(+) ±4.213E+01	6.886E+03(+) ±2.674E+02	4.789E+04(+) ±5.721E+03	4.050E+04(+) ±3.281E+03
3	**2.007E+01 ±4.182E-02**	2.036E+01(+) ±2.289E-02	2.162E+01(+) ±4.061E-03	2.161E+01(+) ±2.374E-01	2.161E+01(+) ±4.197E-02	2.11E+01(+) ±2.394E-02	2.172E+01(+) ±3.486E-02	2.171E+01(+) ±7.976E-01	2.165E+01(+) ±7.686E-02
*4	**8.932E+09 ±6.812E+08**	1.902E+11(+) ±8.122E+10	3.502E+10(+) ±3.645E+09	2.603E+12(+) ±1.218E+12	2.681E+12(+) ±7.639E+11	1.632E+11(+) ±2.938E+10	1.220E+11(+) ±8.678E+10	2.440E+12(+) ±5.409E+11	5.084E+12(+) ±5.728E+11
*5	5.386E+06 ±1.982E+06	9.455E+06(+) ±2.514E+06	**1.079E+06(-) ±1.485E+05**	1.172E+06(-) ±1.845E+05	4.205E+06(≈) ±2.954E+05	2.274E+06(-) ±4.255E+05	9.370E+06(+) ±1.126E+05	1.534E+06(-) ±5.214E+06	4.976E+07(+) ±2.386E+06
6	1.048E+06 ±2.147E+03	1.056E+06(≈) ±4.632E+03	1.066E+06(+) ±7.156E+02	1.065E+06(+) ±8.675E+02	1.062E+06(+) ±1.287E+03	1.078E+06(+) ±2.745E+03	1.063E+06(+) ±6.845E+03	**1.000E+06(-) ±8.232E+03**	1.071E+06(+) ±3.219E+04
7	**1.3506E+07 ±7.215E+06**	3.268E+09(+) ±2.404E+09	2.140E+07(+) ±8.465E+06	4.442E+09(+) ±1.532E+09	6.739E+08(+) ±2.035E+08	1.756E+09(+) ±7.201E+08	2.871E+07(+) ±4.212E+06	3.240E+14(+) ±7.248E+13	1.210E+15(+) ±5.317E+10
*8	2.299E+15 ±2.047E+15	1.547E+16(+) ±7.053E+15	3.488E+15(+) ±2.872E+15	2.885E+15(≈) ±1.413E+15	6.162E+16(+) ±1.492E+16	3.184E+15(+) ±7.612E+14	**2.189E+15(≈) ±1.062E+15**	1.940E+16(+) ±1.382E+16	1.460E+17(+) ±2.014E+15
*9	5.818E+08 ±1.195E+08	8.084E+08(+) ±1.636E+08	6.199E+08(+) ±2.793E+07	6.007E+08(+) ±4.263E+07	6.812E+08(+) ±1.751E+07	3.890E+08(-) ±3.527E+07	**3.020E+08(-) ±3.214E+07**	3.270E+09(+) ±6.832E+07	3.730E+09(+) ±5.833E+08
10	**9.375E+07 ±5.623E+05**	9.423E+07(≈) ±6.224E+05	9.464E+07(≈) ±2.725E+05	9.523E+07(+) ±1.332E+05	9.464E+07(≈) ±1.426E+05	9.447E+07(≈) 1.012E+05	9.829E+07(+) ±3.321E+05	9.803E+07(+) 3.115E+05	9.696E+07(+) ±2.133E+05
11	8.243E+09 ±6.352E+09	2.297E+11(+) ±6.313E+10	2.998E+17(+) ±1.245E+15	6.327E+17(+) ±8.848E+14	2.364E+10(+) ±4.967E+09	1.850E+10(+) ±2.786E+08	**1.720E+09(-) ±2.179E+08**	6.630E+22(+) ±8.795E+21	6.470E+16(+) ±2.545E+15
*12	**1.035E+03 ±4.072E+02**	1.766E+05(+) ±1.484E+04	2.497E+05(+) ±6.373E+04	1.079E+03(≈) ±5.374E+01	1.103E+03(≈) ±8.365E+01	1.068E+03(≈) ±1.476E+02	1.079E+03(≈) ±1.343E+02	6.860E+12(+) ±3.215E+12	1.040E+08(+) ±2.354E+07
*13	1.298E+10 ±2.643E+09	2.730E+10(+) ±8.185E+09	2.332E+11(+) ±4.099E+10	3.274E+12(+) ±2.203E+12	7.857E+09(-) ±1.964E+09	1.698E+10(≈) ±1.034E+10	**8.330E+08(-) ±2.573E+08**	2.100E+21(+) ±7.595E+20	9.440E+16(+) ±5.221E+15
14	4.453E+09 ±3.895E+09	3.761E+11(+) ±1.856E+11	6.072E+11(+) ±1.253E+11	2.953E+13(+) ±2.512E+13	5.600E+09(+) ±1.545E+09	2.081E+10(+) ±1.461E+10	1.110E+10(+) ±1.496E+08	**8.990E+08(-) ±7.818E+21**	5.350E+22(+) ±2.527E+17
15	**3.090E+07 ±5.235E+06**	3.306E+07(≈) ±4.994E+06	1.508E+08(+) ±1.593E+07	9.612E+07(+) ±1.317E+07	9.565E+07(+) ±1.628E+07	4.67E+08(+) ±7.041E+06	3.760E+07(+) ±2.507E+06	8.230E+15(+) ±6.947E+14	2.350E+15(+) ±5.319E+14
	NA	12/3/0	13/1/1	12/2/1	11/3/1	9/2/3	10/2/3	12/0/3	15/0/0

5.2 Comparison Analysis

5.2.1 Comparison With Other Algorithms. Table 2 presents the mean optimization results from 25 independent runs for each algorithm. The symbols "+", "-", and "≈" denote the outcomes of the Wilcoxon rank-sum test at the 0.05 significance level, indicating whether the competing method performed better (+), worse (-), or showed no significant difference (≈) compared to LCC-CMAES. The last column shows the test results for each algorithm, listing the number of times LCC-CMAES significantly outperformed competitors, instances with no significant difference, and cases where LCC-CMAES performed worse.

Based on the results from Table 2, we can analyze the following outcomes:

- **Superior Performance Within CC Frameworks**: LCC-CMAES demonstrates significant advantages compared to existing advanced algorithms within the CC framework. LCC-CMAES shows pronounced superiority on more challenging grouping problems (such as F12-F14 Overlapping Functions), highlighting its capability to handle more complex real-world problems.
- **Generalization Capability**: LCC-CMAES exhibits a degree of generalizability, thanks to the well-designed state that provides it with ample information. It shows certain generalization on similar problem types, such as F11, achieving commendable results when trained on F8-F9. Moreover, LCC-CMAES also demonstrated generalizability on completely unseen problem types (F15). This reveals LCC's ability to solve more complex real-world problems.
- **Improvement Over CC-CMAES**: LCC-CMAES also shows significant improvements compared to CC-CMAES, which employs the same decomposition strategy pool. This improvement is attributed to our effective design of the reward and state, which encourage more rational decomposition strategies and superior outcomes.
- **Comparison of Baselines Without CC**: LCC-CMAES outperforms CMA-ES, LM-MA-ES, MetaES, and L-BFGS, demonstrating its effectiveness.

Table 3: Comparison of resource consumption.

Algorithm	LCC-CMAES	CC-CMAES	CSG	FII	CMA-ES
FEs	**0**	**0**	4.86E+04	4.52E+03	**0**
Time Cost (s)	85.12	**83.88**	275.05	264.81	450.4

5.2.2 Comparison of Resource Consumption. In the analysis above, LCC-CMAES demonstrated its capability to handle more complex problems with a small budget. Besides the additional FEs, the time cost for decomposition and optimization is another bottleneck that constrains the algorithm from being applied to more complex and higher-dimensional problems. Therefore, we conducted a more thorough investigation into the additional FEs and the time cost for decomposition and optimization of each algorithm, aiming for a more detailed presentation of resource consumption.

Table 3 shows the additional FEs for decomposition and the averaged optimization time cost (in seconds) for each problem and each run. To avoid the issue of additional FEs becoming excessively high due to certain problems being difficult to decompose, we use the median to reflect the additional FEs. A notable advantage of LCC-CMAES is that it does not require additional FEs for decomposition, and due to its simple actor and critic network design, the time expenditure for LCC-CMAES is relatively low. This provides a feasible

Table 4: Comparing LCC-CMAES with comparison algorithms on BNS.

problem	LCC-CMAES	CC-CMAES	CSG	FII	CMA-ES	Sep-CMAES	LM-MA-ES
1	**0.000E+00** ±0.000E+00	1.901E-07(+) ±1.523E-07	1.741E-06(+) ±9.037E-07	7.022E-04(+) ±8.325E-04	4.924E-11(+) ±6.043E-11	**0.000E+00(≈)** ±0.000E+00	**0.000E+00(≈)** ±0.000E+00
2	7.044E+00 ±4.794E-01	**6.947E+00(≈)** ±2.413E+00	1.035E+01(+) ±1.532E+00	1.432E+01(+) ±4.413E+00	1.038E+01(+) ±4.152E-01	2.375E+02(+) ±6.350E+01	8.306E+03(+) ±4.066E+03
3	7.292E+05 ±3.484E+05	**5.375E+05(-)** ±2.567E+05	7.173E+05(≈) ±3.925E+05	7.136E+05(≈) ±3.935E+05	8.565E+05(+) ±1.889E+05	8.361E+05(+) ±4.656E+05	3.472E+06(+) ±2.455E+05
4	**1.423E+09** ±3.39E+08	4.124E+09(+) ±1.484E+08	2.545E+10(+) ±8.115E+08	8.347E+10(+) ±4.438E+09	1.852E+10(+) ±6.297E+08	3.865E+09(+) ±2.779E+08	7.713E+09(+) ±2.510E+08
5	**0.000E+00** ±0.000E+00	4.402E-08(+) ±4.876E-08	4.414E-02(+) ±5.875E-03	2.624E-02(+) ±5.914E-03	4.225E-11(+) ±6.183E-11	**0.000E+00(≈)** ±0.000E+00	**0.000E+00(≈)** ±0.000E+00
6	**2.105E+01** ±3.663E+00	4.084E+01(+) ±9.414E+00	2.278E+03(+) ±1.385E+03	2.366E+04(+) ±7.106E+03	3.482E+01(+) ±9.285E+00	2.763E+02(+) ±2.884E+01	4.035E+01(+) ±5.894E+00
7	3.841E+06 ±3.052E+05	3.846E+06(≈) ±9.843E+04	5.554E+06(+) ±5.645E+04	6.376E+06(+) ±4.795E+04	**9.947E+05(-)** ±1.596E+05	2.673E+06(-) ±2.435E+05	2.829E+06(-) ±1.932E+05
8	1.789E+10 ±1.124E+09	2.423E+10(+) ±1.523E+09	1.844E+11(+) ±1.804E+10	1.137E+11(+) ±5.171E+09	1.932E+10(≈) ±4.945E+08	2.674E+10(+) ±1.142E+09	**8.473E+09(-)** ±5.432E+08
9	**0.000E+00** ±0.000E+00	2.094E-07(+) ±1.467E-07	7.395E-04(+) ±2.178E-04	3.975E-02(+) ±4.973E-03	4.452E-08(+) ±1.775E-08	**0.000E+00(≈)** ±0.000E+00	**0.000E+00(≈)** ±0.000E+00
10	3.074E+02 ±2.901E+02	4.213E+04(+) ±3.372E+04	3.725E+04(+) ±9.991E+03	1.713E+05(+) ±7.537E+04	**1.062E+01(-)** ±8.083E-01	2.943E+02(≈) ±9.455E+01	5.309E+01(-) ±1.618E+01
11	3.762E+06 ±2.562E+05	4.062E+06(+) ±1.235E+05	3.506E+06(≈) ±7.774E+04	7.196E+06(+) ±4.051E+04	4.574E+06(+) ±2.962E+05	**3.475E+06(≈)** ±5.456E+05	6.021E+06(+) ±6.778E+04
12	**1.821E+10** ±3.413E+09	4.289E+10(+) ±2.254E+09	5.365E+10(+) ±1.965E+09	2.075E+10(+) ±1.494E+09	1.911E+10(+) ±7.118E+08	3.913E+10(+) ±2.371E+09	2.782E+10(+) ±2.407E+09
	NA	9/2/1	10/2/0	11/1/0	9/1/2	6/5/1	6/3/3

approach for solving more complex and higher-dimensional real-world problems. Other algorithms under CC framework often suffer from excessive costs due to the difficulty in identifying separable types; even when problems are identifiable, the presence of overlapping issues can result in many subgroups remaining too large, thereby causing substantial time expenditures for the underlying optimizer. Besides, we note that the algorithm without CC usually requires more optimization time than CC methods. For instance, CC methods based on CMA-ES are faster than CMA-ES without CC. It is contributed by the decomposed smaller subspace dimensions, which validates the necessity of problem decomposition and CC.

5.3 The Transferability Study

To more effectively test the transferability of LCC-CMAES, we tested using an entirely new set of problems that it had never encountered before with the same settings. The majority of separable functions in the CEC 2013 LSGO are additively separable, with the Ackley function being the only non-additively separable function among the basic functions. To address these limitations, the BNS [5] introduces four non-additively separable base functions, including two multiplicatively separable base functions and two composite separable base functions. Based on these basis functions, BNS designs 12 test problems with varying degrees of separability. Compared to CEC 2013 LSGO, the problems in BNS are closer to the potential complex problems encountered in real-world scenarios. All algorithm settings are consistent with those used in the comparison in CEC2013 LSGO.

Table 4 presents the comparative results, indicating that LCC-CMAES has certain advantages. On one hand, some algorithms consume excessively high additional FEs for decomposition on BNS problems, which prevents them from focusing resources on the optimization process, leading to poor performance (e.g., CSG). On the other hand, some algorithms fail to correctly identify variable interactions and group them on more complex problems, resulting in poor performance (e.g., FII). These results also reveal the transferability of LCC-CMAES and its potential in tackling more complex real-world problems: trained on simpler benchmarks, it can transfer decomposition knowledge to more complex real scenarios, discovering grouping structures through learning rather than relying on expert-level knowledge. However, LSGO variant algorithms and CMA-ES don't encounter issues with inaccurate decomposition and additional FEs, and it can fully leverage the relationships between variables. This underscores the challenge that CC faces in thousands to low dimensions compared to global optimizers.

5.4 Comparison on Neuroevolution tasks

In this section, we adopt four Neuroevolution [63] tasks as a showcase on real-world applications, in which optimization algorithms are used to evolve a population of neural networks according to their performance on a specific machine learning task such as robotic control [12]. Concretely, we consider the real-parameter optimization of 2-layer MLPs for 4 Mujoco [71] robot control tasks: *InvertedDoublePendulum-v4*, *HalfCheetah-v4*, *Pusher-v4* and *Ant-v4*. We set the hidden dimensions of the MLPs to 64 with Tanh activation function, while the input and output dimensions match the control protocols of the Mujoco tasks, leading to 833, 1542, 1991 and 2312 dimensions for the four MLPs of the four tasks, respectively. Because the evaluations of the networks are time consuming, we set the maximum function evaluations (FEs) of all tasks to 1,000. For baselines, we zero-shot the pre-trained LCC-CMAES agent in Section 5.2 to the Neuroevolution problems. The CC-based baselines except CC-CMAES fail to decompose the problem dimensions within 1,000 FEs so we do not include them in the comparison. The hyper-parameter settings of LCC-CMAES and included baselines are consistent with Section 5.1.2 except the total number of generations (TG) and the offspring size (λ) of LCC-CMAES which are both set to 5. Since the targets in the Mujoco tasks are to maximize the

Table 5: Comparison results on Neuroevolution tasks.

	LCC-CMAES	CC-CMAES	CMA-ES	Sep-CMAES	LM-MA-ES	MetaES	L-BFGS
InvertedDoublePendulum-v4 (833D)	**-5.111E+03 ±2.914E+02**	-4.854E+03 (+) ±2.354E+02	-4.714E+03 (+) ±2.354E+02	-4.988E+03 (+) ±2.644E+02	-4.951E+03 (+) ±2.455E+02	-4.596E+03 (+) ±2.944E+02	-4.322E+03 (+) ±2.831E+02
HalfCheetah-v4 (1542D)	**-2.451E+02 ±2.514E+02**	-2.017E+02 (+) ±2.119E+02	-1.914E+02 (+) ±1.645E+02	-1.849E+02 (+) ±2.002E+02	-1.744E+02 (+) ±1.988E+02	-5.554E+01 (+) ±9.624E+01	-4.997E+01 (+) ±9.487E+01
Pusher-v4 (1991D)	**3.354E+02 ±2.984E+01**	3.543E+02 (+) ±3.791E+01	3.497E+02 (+) ±4.016E+01	3.481E+02 (+) ±3.594E+01	3.411E+02 (+) ±2.746E+01	3.894E+02 (+) ±4.687E+01	3.909E+02 (+) ±6.314E+01
Ant-v4 (2312D)	**-1.083E+03 ±6.476E+01**	-1.080E+03 (≈) ±6.524E+01	-1.073E+03 (+) ±5.146E+01	-1.066E+03 (+) ±5.687E+01	-1.079E+03 (≈) ±5.377E+01	-1.015E+03 (+) ±3.345E+01	-1.004E+03 (+) ±2.154E+01
	NA	3/1/0	4/0/0	4/0/0	3/1/0	4/0/0	4/0/0

accumulated rewards gained by the networks, in Table 5 we present the negative accumulated rewards obtained by the networks optimized by LCC-CMAES and baselines to keep the minimization optimization manner.

The results show that even on realistic optimization problems with larger problem dimensions and more complex variable relationship, our LCC-CMAES still retains its advantages over CC-CMAES and global optimization baselines, validating its effectiveness.

5.5 Ablation Study

5.5.1 State features Study. To verify the necessity of the framework components, we conducted ablation studies on the state features. Specifically, we separately removed the embeddings and concatenations of the s_{AH} (denoted as W/O AH), s_{GO} (W/O GO), and s_{SD} (W/O SD). We then tested these modifications on the CEC 2013 LSGO under the same settings otherwise unchanged.

The results are shown in Figure 4 (A). For each problem, we use the average performance of all runs for each algorithm and conduct the min-max normalization over all algorithms to restrict their performance into $[0, 1]$ and eliminate the cost scale gaps between different problems. The 1− mean performance over all problems of each algorithms and their error bars are presented, where the higher is better.

It is evident that the performance significantly deteriorates when these features are removed, highlighting their crucial roles to provide sufficient information being available to the RL agent.

5.5.2 Reward Study. The design of the reward mechanism needs to employ a ratio-based approach within the range of -1 to 1 to address that evaluation values can vary significantly across different problems and excessive impacts on network training. Here are two other reward designs that meet these requirements:

1) Global best fitness descent ratio: The global best fitness is calculated by subtracting it from the initial generation global best fitness and normalizing the decline by the global best fitness of the initial generation. reward1 := $r_t = \frac{f_0^* - f_t^*}{f_0^*}$

2) Relative global best fitness descent ratio: The global best fitness decline normalized by the global best fitness in previous generation. reward2 := $r_t = \frac{f_{t-1}^* - f_t^*}{f_{t-1}^*}$

Figure 4 (B) also presents the results under different reward schemes, with the same normalization as introduced in Section 5.5.1. It can be observed that the reward1 and the reward2 are significantly less effective than the approach we have adopted. This ineffectiveness is due to the fact that using a scheme that subtracts the initial generation's fitness can lead to many subsequent fitness

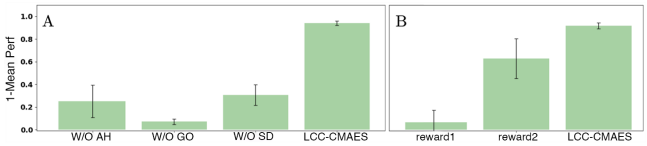

Figure 4: The ablation study on state features and reward designs.

values significantly lower than the initial generation, making the numerator approximately equal to the initial value, causing the formula to approach 1. Additionally, normalizing the global best fitness from the previous generation causes the standard for reward normalization to change with each generation. This inconsistency makes it challenging for the RL agent to select appropriate actions.

6 Conclusion and Future work

We have proposed LCC, a pioneering learning-based cooperative coevolution framework that dynamically schedules decomposition strategies during optimization processes. With CMA-ES as the underlying optimizer, we instantiate LCC, naming it the LCC-CMAES algorithm. Unlike previous algorithms under the CC framework, LCC-CMAES does not use expert-designed knowledge for decomposition but instead utilizes statistical features for DRL to select most-expected decomposition strategies. More importantly, LCC-CMAES does not require the additional FEs for decomposition, allowing it to focus resources on optimization. When tested against several other advanced algorithms on two benchmarks, CEC 2013 LSGO and BNS, the comparative results demonstrated that LCC-CMAES holds a distinct advantage, especially on complex real-world problems that it had not previously encountered. This underscores LCC's robustness, adaptability and transferability, making it a promising approach for tackling complex optimization challenges in various settings.

Looking ahead to future work, we hope to: (1) investigate the inclusion of more complex or higher-dimensional features that may capture deeper insights into the problem's structure; (2) design more rational and effective decomposition actions. These goals aim to refine LCC's effectiveness and applicability, ensuring it can be a versatile tool in the LSGO, capable of addressing a broader range of complex challenges.

References

[1] Youhei Akimoto and Nikolaus Hansen. 2016. Projection-based restricted covariance matrix adaptation for high dimension. In *Proceedings of the Genetic and Evolutionary Computation Conference 2016*. 197–204.

[2] Maumita Bhattacharya, Rafiqul Islam, and Jemal Abawajy. 2016. Evolutionary optimization: a big data perspective. *Journal of Network and Computer Applications* 59 (2016), 416–426.

[3] Richard H Byrd, Peihuang Lu, Jorge Nocedal, and Ciyou Zhu. 1995. A limited memory algorithm for bound constrained optimization. *SIAM Journal on scientific computing* 16, 5 (1995), 1190–1208.

[4] Jiacheng Chen, Zeyuan Ma, Hongshu Guo, Yining Ma, Jie Zhang, and Yue-jiao Gong. 2024. Symbol: Generating Flexible Black-Box Optimizers through Symbolic Equation Learning. *arXiv preprint arXiv:2402.02355* (2024).

[5] Minyang Chen, Wei Du, Yang Tang, Yaochu Jin, and Gary G Yen. 2022. A decomposition method for both additively and non-additively separable problems. *IEEE Transactions on Evolutionary Computation* (2022).

[6] Minyang Chen, Chenchen Feng, and Ran Cheng. 2025. MetaDE: Evolving Differential Evolution by Differential Evolution. *IEEE Transactions on Evolutionary Computation* (2025).

[7] Wei-Neng Chen, Ya-Hui Jia, Feng Zhao, Xiao-Nan Luo, Xing-Dong Jia, and Jun Zhang. 2019. A cooperative co-evolutionary approach to large-scale multisource water distribution network optimization. *IEEE Transactions on Evolutionary Computation* 23, 5 (2019), 842–857.

[8] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, et al. 2024. Symbolic discovery of optimization algorithms. *Advances in Neural Information Processing Systems* 36 (2024).

[9] Géremi Gilson Dranka, Paula Ferreira, and A Ismael F Vaz. 2021. A review of co-optimization approaches for operational and planning problems in the energy sector. *Applied Energy* 304 (2021), 117703.

[10] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. 2019. Neural architecture search: A survey. *Journal of Machine Learning Research* 20, 55 (2019), 1–21.

[11] Maxence Faldor, Robert Tjarko Lange, and Antoine Cully. 2025. Discovering Quality-Diversity Algorithms via Meta-Black-Box Optimization. *arXiv preprint arXiv:2502.02190* (2025).

[12] Edgar Galván and Peter Mooney. 2021. Neuroevolution in deep neural networks: Current trends and future challenges. *IEEE Transactions on Artificial Intelligence* 2, 6 (2021), 476–493.

[13] Hongwei Ge, Liang Sun, Xin Yang, Shinichi Yoshida, and Yanchun Liang. 2015. Cooperative differential evolution with fast variable interdependence learning and cross-cluster mutation. *Applied Soft Computing* 36 (2015), 300–314.

[14] Riccardo Guidotti, Anna Monreale, Salvatore Ruggieri, Franco Turini, Fosca Giannotti, and Dino Pedreschi. 2018. A Survey of Methods for Explaining Black Box Models. *ACM Comput. Surv.* 51, 5, Article 93 (aug 2018), 42 pages. doi:10.1145/3236009

[15] Hongshu Guo, Sijie Ma, Zechuan Huang, Yuzhi Hu, Zeyuan Ma, Xinglin Zhang, and Yue-Jiao Gong. 2025. Reinforcement Learning-based Self-adaptive Differential Evolution through Automated Landscape Feature Learning. In *Proceedings of the Genetic and Evolutionary Computation Conference*.

[16] Hongshu Guo, Yining Ma, Zeyuan Ma, Jiacheng Chen, Xinglin Zhang, Zhiguang Cao, Jun Zhang, and Yue-Jiao Gong. 2024. Deep Reinforcement Learning for Dynamic Algorithm Selection: A Proof-of-Principle Study on Differential Evolution. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* (2024).

[17] Hongshu Guo, Zeyuan Ma, Jiacheng Chen, Yining Ma, Zhiguang Cao, Xinglin Zhang, and Yue-Jiao Gong. 2025. ConfigX: Modular Configuration for Evolutionary Algorithms via Multitask Reinforcement Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 26982–26990.

[18] PC Hammer. 1962. Adaptive control processes: a guided tour (R. Bellman).

[19] Nikolaus Hansen. 2016. The CMA evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772* (2016).

[20] Xiaoyu He, Zibin Zheng, and Yuren Zhou. 2020. MMES: Mixture model-based evolution strategy for large-scale optimization. *IEEE Transactions on Evolutionary Computation* 25, 2 (2020), 320–333.

[21] Ya-Hui Jia, Yi Mei, and Mengjie Zhang. 2020. Contribution-based cooperative co-evolution for nonseparable large-scale problems with overlapping subcomponents. *IEEE Transactions on Cybernetics* 52, 6 (2020), 4246–4259.

[22] Marcin Michal Komarnicki, Michal Witold Przewozniczek, Renato Tinós, and Xiaodong Li. 2024. Overlapping Cooperative Co-Evolution for Overlapping Large-Scale Global Optimization Problems. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 665–673.

[23] Robert Lange, Tom Schaul, Yutian Chen, Tom Zahavy, Valentin Dalibard, Chris Lu, Satinder Singh, and Sebastian Flennerhag. 2023. Discovering evolution strategies via meta-black-box optimization. In *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*. 29–30.

[24] Jian-Yu Li, Zhi-Hui Zhan, Kay Chen Tan, and Jun Zhang. 2022. Dual differential grouping: A more general decomposition method for large-scale optimization. *IEEE Transactions on Cybernetics* (2022).

[25] Pengyi Li, Jianye Hao, Hongyao Tang, Xian Fu, Yan Zhen, and Ke Tang. 2024. Bridging evolutionary algorithms and reinforcement learning: A comprehensive survey on hybrid algorithms. *IEEE Transactions on Evolutionary Computation* (2024).

[26] Xiaodong Li, Ke Tang, Mohammad N Omidvar, Zhenyu Yang, Kai Qin, and Hefei China. 2013. Benchmark functions for the CEC 2013 special session and competition on large-scale global optimization. *Gene* 7, 33 (2013), 8.

[27] Xiaobin Li, Kai Wu, Xiaoyu Zhang, and Handing Wang. 2025. B2Opt: Learning to optimize black-box optimization with little budget. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 18502–18510.

[28] Xiaobin Li, Kai Wu, Xiaoyu Zhang, Handing Wang, Jing Liu, et al. 2024. Pretrained optimization model for zero-shot black box optimization. *Advances in Neural Information Processing Systems* 37 (2024), 14283–14324.

[29] Zhenhua Li and Qingfu Zhang. 2017. A simple yet efficient evolution strategy for large-scale black-box optimization. *IEEE Transactions on Evolutionary Computation* 22, 5 (2017), 637–646.

[30] Hongqiao Lian, Zeyuan Ma, Hongshu Guo, Ting Huang, and Yue-Jiao Gong. 2024. RLEMMO: Evolutionary Multimodal Optimization Assisted By Deep Reinforcement Learning. In *Proceedings of the Genetic and Evolutionary Computation Conference*.

[31] Zuowen Liao, Wenyin Gong, and Shuijia Li. 2023. Two-stage reinforcement learning-based differential evolution for solving nonlinear equations. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* (2023).

[32] Jing Liu, Ruhul Sarker, Saber Elsayed, Daryl Essam, and Nurhadi Siswanto. 2024. Large-scale evolutionary optimization: A review and comparative study. *Swarm and Evolutionary Computation* (2024), 101466.

[33] Jinpeng Liu and Ke Tang. 2013. Scaling up covariance matrix adaptation evolution strategy using cooperative coevolution. In *International Conference on Intelligent Data Engineering and Automated Learning*. Springer, 350–357.

[34] Ilya Loshchilov. 2017. LM-CMA: An alternative to L-BFGS for large-scale black box optimization. *Evolutionary computation* 25, 1 (2017), 143–171.

[35] Ilya Loshchilov, Tobias Glasmachers, and Hans-Georg Beyer. 2018. Large scale black-box optimization by limited-memory matrix adaptation. *IEEE Transactions on Evolutionary Computation* 23, 2 (2018), 353–358.

[36] Zeyuan Ma, Jiacheng Chen, Hongshu Guo, Yining Ma, and Yue-Jiao Gong. 2024. Auto-configuring Exploration-Exploitation Tradeoff in Evolutionary Computation via Deep Reinforcement Learning. In *Proceedings of the Genetic and Evolutionary Computation Conference*.

[37] Zeyuan Ma, Yue-Jiao Gong, Hongshu Guo, Wenjie Qiu, Sijie Ma, Hongqiao Lian, Jiajun Zhan, Kaixu Chen, Chen Wang, Zhiyang Huang, et al. 2025. MetaBox-v2: A Unified Benchmark Platform for Meta-Black-Box Optimization. *arXiv preprint arXiv:2505.17745* (2025).

[38] Zeyuan Ma, Hongshu Guo, Yue-Jiao Gong, Jun Zhang, and Kay Chen Tan. 2024. Toward Automated Algorithm Design: A Survey and Practical Guide to Meta-Black-Box-Optimization. *arXiv preprint arXiv:2411.00625* (2024).

[39] Zeyuan Ma, Zhiyang Huang, Jiacheng Chen, Zhiguang Cao, and Yue-Jiao Gong. 2025. Surrogate Learning in Meta-Black-Box Optimization: A Preliminary Study. In *Proceedings of the Genetic and Evolutionary Computation Conference*.

[40] Zeyuan Ma, Hongqiao Lian, Wenjie Qiu, and Yue-Jiao Gong. 2025. Accurate Peak Detection in Multimodal Optimization via Approximated Landscape Learning. In *Proceedings of the Genetic and Evolutionary Computation Conference*.

[41] Yi Mei, Mohammad Nabi Omidvar, Xiaodong Li, and Xin Yao. 2016. A competitive divide-and-conquer algorithm for unconstrained large-scale black-box optimization. *ACM Transactions on Mathematical Software (TOMS)* 42, 2 (2016), 1–24.

[42] Olaf Mersmann, Bernd Bischl, Heike Trautmann, Mike Preuss, Claus Weihs, and Günter Rudolph. 2011. Exploratory landscape analysis. In *Proceedings of the 13th Annual Conference on Genetic and Evolutionary Computation*. 829–836.

[43] Mohamed Meselhi, Ruhul Sarker, Daryl Essam, and Saber Elsayed. 2022. A decomposition approach for large-scale non-separable optimization problems. *Applied Soft Computing* 115 (2022), 108168.

[44] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. 2015. Human-level control through deep reinforcement learning. *nature* 518, 7540 (2015), 529–533.

[45] Shibing Mo, Kai Wu, Qixuan Gao, Xiangyi Teng, and Jing Liu. 2025. AutoSGNN: automatic propagation mechanism discovery for spectral graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 19493–19502.

[46] Vinod Nair and Geoffrey E Hinton. 2010. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th international conference on machine learning (ICML-10)*. 807–814.

[47] Mohammad Nabi Omidvar, Xiaodong Li, Zhenyu Yang, and Xin Yao. 2010. Cooperative co-evolution for large scale optimization through more frequent random grouping. In *2010 IEEE Congress on Evolutionary Computation(CEC)*. IEEE, 1–8.

[48] Mohammad Nabi Omidvar, Xiaodong Li, and Xin Yao. 2021. A review of population-based metaheuristics for large-scale black-box global optimization—Part I. *IEEE Transactions on Evolutionary Computation* 26, 5 (2021), 802–822.

[49] Mohammad Nabi Omidvar, Xiaodong Li, and Xin Yao. 2021. A review of population-based metaheuristics for large-scale black-box global optimization—Part II. *IEEE Transactions on Evolutionary Computation* 26, 5 (2021), 823–843.

[50] Mohammad Nabi Omidvar, Ming Yang, Yi Mei, Xiaodong Li, and Xin Yao. 2017. DG2: A faster and more accurate differential grouping for large-scale black-box optimization. *IEEE Transactions on Evolutionary Computation* 21, 6 (2017), 929–942.

[51] Erik Pitzer and Michael Affenzeller. 2012. A comprehensive survey on fitness landscape analysis. *Recent Advances in Intelligent Engineering Systems* (2012), 161–191.

[52] Mitchell A Potter and Kenneth A De Jong. 1994. A cooperative coevolutionary approach to function optimization. In *International conference on parallel problem solving from nature*. Springer, 249–257.

[53] Wenjie Qiu, Hongshu Guo, Zeyuan Ma, and Yue-Jiao Gong. 2025. A Novel Two-Phase Cooperative Co-evolution Framework for Large-Scale Global Optimization with Complex Overlapping. In *Proceedings of the Genetic and Evolutionary Computation Conference*.

[54] Raymond Ros and Nikolaus Hansen. 2008. A simple modification in CMA-ES achieving linear time and space complexity. In *International conference on parallel problem solving from nature*. Springer, 296–305.

[55] Rajkumar Roy and Ashutosh Tiwari. 2002. Generalised regression GA for handling inseparable function interaction: Algorithm and applications. In *International Conference on Parallel Problem Solving from Nature*. Springer, 452–461.

[56] Ralf Salomon. 1996. Re-evaluating genetic algorithm performance under coordinate rotation of benchmark functions. A survey of some theoretical and practical aspects of genetic algorithms. *BioSystems* 39, 3 (1996), 263–278.

[57] Eman Sayed, Daryl Essam, and Ruhul Sarker. 2012. Dependency identification technique for large scale optimization problems. In *2012 IEEE Congress on Evolutionary Computation(CEC)*. IEEE, 1–8.

[58] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).

[59] Shuai Shao, Ye Tian, and Yajie Zhang. 2025. Deep reinforcement learning assisted surrogate model management for expensive constrained multi-objective optimization. *Swarm and Evolutionary Computation* 92 (2025), 101817.

[60] Mudita Sharma, Alexandros Komninos, Manuel López-Ibáñez, and Dimitar Kazakov. 2019. Deep reinforcement learning based parameter control in differential evolution. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 709–717.

[61] Yan-jun Shi, Hong-fei Teng, and Zi-qiang Li. 2005. Cooperative co-evolutionary differential evolution for function optimization. In *Advances in Natural Computation: First International Conference, ICNC 2005, Changsha, China, August 27-29, 2005, Proceedings, Part II 1*. Springer, 1080–1088.

[62] IM Sobol'. 1993. Sensitivity estimates for nonlinear mathematical models. *Math. Model. Comput. Exp.* 1 (1993), 407.

[63] Felipe Petroski Such, Vashisht Madhavan, Edoardo Conti, Joel Lehman, Kenneth O Stanley, and Jeff Clune. 2017. Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning. *arXiv preprint arXiv:1712.06567* (2017).

[64] Jianyong Sun, Xin Liu, Thomas Bäck, and Zongben Xu. 2021. Learning adaptive differential evolution algorithm from optimization experiences by policy gradient. *IEEE Transactions on Evolutionary Computation* 25, 4 (2021), 666–680.

[65] Yuan Sun, Michael Kirley, and Saman K Halgamuge. 2017. A recursive decomposition method for large scale continuous optimization. *IEEE Transactions on Evolutionary Computation* 22, 5 (2017), 647–661.

[66] Zhiping Tan and Kangshun Li. 2021. Differential evolution with mixed mutation strategy based on deep reinforcement learning. *Applied Soft Computing* 111 (2021), 107678.

[67] Maojiang Tian, Mingke Chen, Wei Du, Yang Tang, and Yaochu Jin. 2024. An Enhanced Differential Grouping Method for Large-Scale Overlapping Problems. *IEEE Transactions on Evolutionary Computation* (2024).

[68] Maojiang Tian, Minyang Chen, Wei Du, Yang Tang, Yaochu Jin, and Gary G Yen. 2024. A Composite Decomposition Method for Large-Scale Global Optimization. *IEEE Transactions on Artificial Intelligence* (2024).

[69] Ashutosh Tiwari and Rajkumar Roy. 2002. Variable dependence interaction and multi-objective optimisation. In *Proceedings of the 4th Annual Conference on Genetic and Evolutionary Computation*. 602–609.

[70] Ashutosh Tiwari, Rajkumar Roy, Graham Jared, and Olivier Munaux. 2001. Interaction and multi-objective optimisation. In *Proceedings of the 3rd Annual Conference on Genetic and Evolutionary Computation*. 671–678.

[71] Emanuel Todorov, Tom Erez, and Yuval Tassa. 2012. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 5026–5033.

[72] Frans Van den Bergh and Andries P Engelbrecht. 2004. A cooperative approach to particle swarm optimization. *IEEE Transactions on Evolutionary Computation* 8, 3 (2004), 225–239.

[73] Paul Vicol, Luke Metz, and Jascha Sohl-Dickstein. 2021. Unbiased gradient estimation in unrolled computation graphs with persistent evolution strategies. In *International Conference on Machine Learning*. PMLR, 10553–10563.

[74] Di Wu and G Gary Wang. 2022. Employing reinforcement learning to enhance particle swarm optimization methods. *Engineering Optimization* 54, 2 (2022), 329–348.

[75] Yue Xu and Dechang Pi. 2020. A reinforcement learning-based communication topology in particle swarm optimization. *Neural Computing and Applications* 32 (2020), 10007–10032.

[76] Ke Xue, Jiacheng Xu, Lei Yuan, Miqing Li, Chao Qian, Zongzhang Zhang, and Yang Yu. 2022. Multi-agent dynamic algorithm configuration. *Advances in Neural Information Processing Systems* 35 (2022), 20147–20161.

[77] Ming Yang, Jie Gao, Aimin Zhou, Changhe Li, and Xin Yao. 2023. Contribution-Based Cooperative Co-Evolution With Adaptive Population Diversity for Large-Scale Global Optimization [Research Frontier]. *IEEE Computational Intelligence Magazine* 18, 3 (2023), 56–68.

[78] Ming Yang, Mohammad Nabi Omidvar, Changhe Li, Xiaodong Li, Zhihua Cai, Borhan Kazimipour, and Xin Yao. 2016. Efficient resource allocation in cooperative co-evolution for large-scale global optimization. *IEEE Transactions on Evolutionary Computation* 21, 4 (2016), 493–505.

[79] Qingyong Yang, Shu-Chuan Chu, Jeng-Shyang Pan, Jyh-Horng Chou, and Junzo Watada. 2024. Dynamic multi-strategy integrated differential evolution algorithm based on reinforcement learning for optimization problems. *Complex & Intelligent Systems* 10, 2 (2024), 1845–1877.

[80] Xu Yang, Rui Wang, and Kaiwen Li. [n. d.]. Meta-Black-Box Optimization for Evolutionary Algorithms: Review and Perspective. *Available at SSRN 4956956* ([n. d.]).

[81] Zhenyu Yang, Ke Tang, and Xin Yao. 2008. Large scale evolutionary optimization using cooperative coevolution. *Information Sciences* 178, 15 (2008), 2985–2999.

[82] Wenjie Yi, Rong Qu, Licheng Jiao, and Ben Niu. 2022. Automated design of metaheuristics using reinforcement learning within a novel general search framework. *IEEE Transactions on Evolutionary Computation* 27, 4 (2022), 1072–1084.

[83] Shiyuan Yin, Yi Liu, GuoLiang Gong, Huaxiang Lu, and Wenchang Li. 2021. RLEPSO: Reinforcement learning based Ensemble particle swarm optimizer. In *Proceedings of the 2021 4th International Conference on Algorithms, Computing and Artificial Intelligence*. 1–6.

[84] Jinlu Zhang, Lixin Wei, Zeyin Guo, Hao Sun, and Ziyu Hu. 2024. A survey of meta-heuristic algorithms in optimization of space scale expansion. *Swarm and Evolutionary Computation* 84 (2024), 101462.

[85] Xin-Yuan Zhang, Yue-Jiao Gong, Ying Lin, Jie Zhang, Sam Kwong, and Jun Zhang. 2019. Dynamic cooperative coevolution for large scale optimization. *IEEE Transactions on Evolutionary Computation* 23, 6 (2019), 935–948.